Manajemen Memori

Week 6: Fragmentasi, Alokasi, & Paging

lectura.id/course/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

OUTLINE

Peta materi pertemuan ini

1. **Algoritma Alokasi Memori**
2. **Fragmentasi Memori**
3. **Teori Paging**
4. **Ringkasan dan Latihan**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Memahami masalah utama dalam alokasi memori dinamis.
2. Menganalisis algoritma alokasi First-Fit, Best-Fit, dan Worst-Fit.
3. Membedakan fragmentasi eksternal dan fragmentasi internal.
4. Memahami konsep dasar Paging sebagai solusi manajemen memori noncontiguous.
5. Menjelaskan cara kerja translasi alamat logis ke fisik pada Paging.

Fokus Kelas Ini

Kita akan mempelajari bagaimana OS mengalokasikan ruang memori, mengelola sisa ruang kosong (fragmentasi), dan teknik perbaikan melalui **Paging**.

Masalah Alokasi Dinamis

Bagaimana menempatkan proses ke memori?

- OS memelihara **tabel memori** yang mencatat blok terpakai dan blok kosong (*hole*).
- Awalnya, memori adalah satu *hole* besar.
- Saat proses-proses mulai berjalan dan berhenti, memori akan berisi kumpulan *holes* berukuran berbeda-beda yang tersebar.

Tantangan Utama

Ketika proses baru datang meminta memori sebesar N, ***hole* mana** dari daftar *free holes* yang harus dipilih OS?

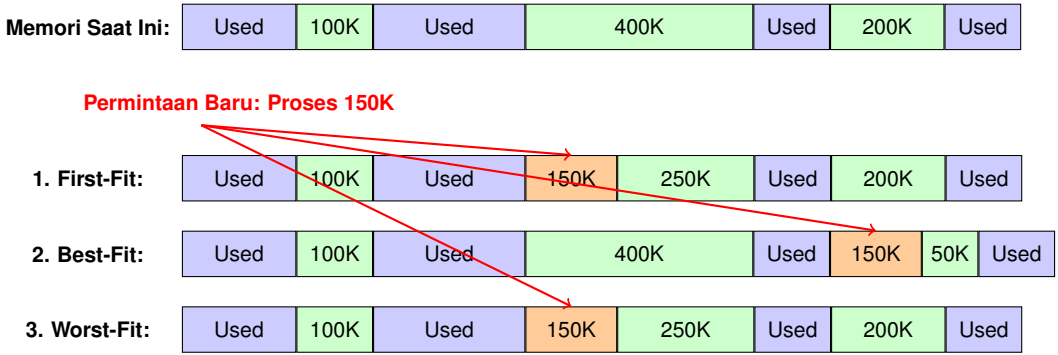
Tiga Strategi Utama Alokasi

Cara OS memilih ruang kosong

Strategi	Cara Kerja	Karakteristik
First-Fit	Pilih *hole* **pertama** yang muat.	Pencarian berhentik cepat. Meninggalkan *hole* kecil di awal memori.
Best-Fit	Pilih *hole* **terkecil** yang muat.	Scan seluruh daftar. Sisa *hole* sangat kecil. Baik untuk menghemat ruang.
Worst-Fit	Pilih *hole* **terbesar** yang muat.	Scan seluruh daftar. Menyisakan *hole* besar. Praktiknya sering kurang performanya.

Ilustrasi Strategi Alokasi

First-fit, Best-fit, Worst-fit



Contoh Soal Strategi Alokasi

Simulasi alokasi blok memori

Soal

Diberikan sekumpulan blok memori kosong (*free holes*) secara berurutan di RAM dengan ukuran: **100K**, **500K**, **200K**, **300K**, dan **600K**.
Terdapat 4 proses baru yang datang dan masing-masing meminta alokasi secara berurutan: **212K**, **417K**, **112K**, dan **624K**.
Tentukan partisi *hole* mana yang akan diisi oleh proses-proses tersebut dengan algoritma:

1. **First-Fit**
2. **Best-Fit**
3. **Worst-Fit**

Jawaban: First-Fit

Langkah-langkah simulasi

Memori Awal: [100K] [500K] [200K] [300K] [600K]

1. **Proses 212K:** First-Fit mencari blok pertama yang $\geq$ 212K.
   - Masuk ke blok **500K**. Sisa blok: 500K - 212K = **288K**.
   - *Status:* [100K] [212K dipakai] [288K sisa] [200K] [300K] [600K]
2. **Proses 417K:** Mencari blok pertama yang $\geq$ 417K.
   - Masuk ke blok **600K**. Sisa blok: 600K - 417K = **183K**.
   - *Status:* [100K] [...] [288K sisa] [200K] [300K] [417K dipakai] [183K sisa]
3. **Proses 112K:** Mencari blok pertama yang $\geq$ 112K.
   - Masuk ke blok sisa **288K**. Sisa blok: 288K - 112K = **176K**.
4. **Proses 624K:** Mencari blok pertama yang $\geq$ 624K.
   - Karena blok sisa tidak ada yang mencukupi, harus **menunggu**.

Jawaban: Best-Fit

Langkah-langkah simulasi

Memori Awal: [100K] [500K] [200K] [300K] [600K]

1. **Proses 212K:** Best-Fit mencari sisa blok terkecil yang $\geq$ 212K (yakni blok 300K).
   - Masuk ke blok **300K**. Sisa blok: 300K - 212K = **88K**.
   - *Status:* [100K] [500K] [200K] [212K dipakai] [88K sisa] [600K]
2. **Proses 417K:** Mencari sisa blok terkecil yang $\geq$ 417K.
   - Masuk ke blok **500K**. Sisa blok: 500K - 417K = **83K**.
3. **Proses 112K:** Mencari sisa blok terkecil yang $\geq$ 112K.
   - Masuk ke blok **200K**. Sisa blok: 200K - 112K = **88K**.
4. **Proses 624K:** Mencari sisa blok terkecil yang $\geq$ 624K.
   - Karena blok sisa tidak ada yang mencukupi (terbesar tinggal 600K, sisa yang lain terlalu kecil), harus **menunggu**.

Jawaban: Worst-Fit

Langkah-langkah simulasi

Memori Awal: [100K] [500K] [200K] [300K] [600K]

1. **Proses 212K:** Worst-Fit mencari sisa blok terbesar yang $\geq$ 212K (yakni blok 600K).
   - Masuk ke blok **600K**. Sisa blok: 600K - 212K = **388K**.
   - *Status:* [100K] [500K] [200K] [300K] [212K dipakai] [388K sisa]
2. **Proses 417K:** Mencari sisa blok terbesar yang $\geq$ 417K.
   - Masuk ke blok **500K**. Sisa blok: 500K - 417K = **83K**.
3. **Proses 112K:** Mencari sisa blok terbesar yang $\geq$ 112K.
   - Masuk ke blok sisa **388K**. Sisa blok: 388K - 112K = **276K**.
4. **Proses 624K:** Mencari sisa blok terbesar yang $\geq$ 624K.
   - Karena blok sisa tidak ada yang mencukupi, harus **menunggu**.

Apa itu Fragmentasi Memori?

Konsekuensi tak terhindarkan dari dinamika OS

Definisi

Fragmentasi adalah kondisi di mana memori total cukup besar, tetapi terpecah menjadi banyak potongan-potongan terpisah yang sulit dimanfaatkan secara efisien.

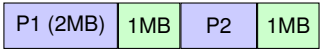
- Muncul sebagai efek samping mekanisme alokasi dan dealokasi memori berulang-ulang.
- Menyebabkan **pemborosan** kapasitas.
- Mengurangi efisiensi penggunaan sumber daya sistem seiring berjalannya operasi OS.

Fragmentasi Eksternal vs Internal

Mengenali dua jenis tipe pemborosan memori

Fragmentasi Eksternal

- Ada ruang memori bebas di **luar** blok per-proses (di antara proses-proses).
- Total *hole* cukup untuk proses baru, namun **tidak berurutan / tidak kontigu**.

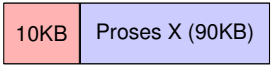


Request 2MB Gagal

Fragmentasi Internal

- Ada ruang memori tak terpakai **di dalam** partisi proses itu sendiri.
- Terjadi bila unit memori yang dialokasikan ukurannya membulat kaku.

Blok Dialokasikan (100KB)



Sisa 10KB Terbuang

Fenomena 50 Percent Rule dan Kompaksi

Karakteristik Fragmentasi dan Solusinya

- **50 Percent Rule**: Secara statistik, untuk strategi alokasi first-fit yang berjalan terus menerus secara acak, diperkirakan $1/3$ **(sepertiga)** blok memori akan terbuang karena fragmentasi eksternal.

Solusi Konvensional: Kompaksi

Mengeser seluruh unit proses yang ada, memadatkannya jadi satu, sehingga semua sisa bebas (holes) memori dapat menyatu besar.

- **Kerugiannya:** Sangat mahal dalam hal **overhead komputasi**. Semua pointer referensi dari sistem memory OS harus dirubah dan dihentikan seketika.

Apa Itu Paging?

Solusi Modern untuk RAM Tidak Bersambung

Definisi Paging

Teknik manajemen memori yang memungkinkan alamat fisik memori dari suatu proses bisa dialokasikan **secara noncontiguous** (tidak berurutan). Teknik Paging menyingkirkan masalah **Fragmentasi Eksternal** sepenuhnya.

- **Memori Fisik** → Dipotong-potong ukurannya identik (blok tetap) yang dinamakan **Frame** (misal: 4KB).
- **Memori Logis / Virtual** → Dipotong-potong ukuran yang *sama persis dengan Frame* dinamakan **Page**.
- **Keuntungan** → Tiap *Page* unik dari sebuah program bebas diselipkan ke sembarang *Frame* kosong manapun di RAM fisik.

Mapping antara Page dan Frame

Peran Page Table

Page Table

- Struktur tabel sentral di OS yang dialokasikan per proses sistem.
- Berfungsi **memetakan** info translasi dari mana **Page Number** ke **Frame Number** yang ada.
- Unit **MMU** (Memory Management Unit) menggunakan tabel ini setiap milidetik secara hardware.

Konsep Kunci

Ukuran *Page* dan ukuran *Frame* **harus persis sama** untuk memastikan proses pemetaan (mapping) berjalan dengan valid dan efisien!

Keunggulan Arsitektur Paging

Konsep Teoritis Paging

Memori Logis (Proses)

- Ruang alamat pengguna/proses dibagi ke *page* ukuran statis.
- Program menganggap memorinya linear dan bersambung.

Memori Fisik (RAM)

- Dibagi ke *frame* ukuran statis.
- Dialokasikan secara acak noncontiguous sesuai ketersediaan.

Mengapa Paging Sangat Baik?

- Benar-benar mengatasi **Fragmentasi Eksternal**.
- Menyederhanakan alokasi: OS bebas menempatkan *page* tanpa mencari blok berurutan.
- Terisolasi dengan baik, dan dapat mendukung penggunaan share memory.

Ringkasan Super Singkat (6 Kalimat Kunci)

Jika lupa materi, ingat 6 poin ini

1. Alokasi memori berurusan dengan metode OS meletakkan blok memori target proses ke celah kosong ruang memori (*hole*).
2. Algoritma *First-fit* paling cepat, sedang *Best-fit* terbaik efisiensinya tapi menumpukkan ruang *fragmented holes*.
3. Pemborosan sisa partisi luasan blok tidak kontigu dinamakan sebagai **Fragmentasi Eksternal**.
4. Konsep teoritis memori modern dengan nama **Paging** digunakan khusus mengelola data proses terpencar (**noncontiguous**).
5. Proses Virtual dieksekusi bernama *Page*, Kapling Fisikal penampung dinamakan *Frame*.
6. Entitas *Page Table* senantiasa dilirik oleh MMU hardware sebagai rujukan menerjemahkan Alamat Maya menuju Alamat RAM Sejati.

Latihan Mandiri

Kerjakan pelan, diskusikan berpasangan

1. Diberitahukan ada sisa luang ruang memori RAM berurutan (*holes*) masing-masing: **15K, 30K, 40K, 25K**.
   Terdapat sebuah Proses C berukuran **22K**. Apakah perbandingan blok partisi tujuan dialokasikan OS apabila OS menerapkan metode strategi implementasi berikut?
   - First-Fit?
   - Best-Fit?
   - Worst-Fit?
2. Mengapa defragmentasi memori (Kompaksi) secara rutin dalam OS modern amat jarang dijalankan / tidak efisien?

Jawaban Latihan Mandiri

Solusi latihan alokasi kustom

1. **Jawaban Alokasi (Kondisi Awal: 15K, 30K, 40K, 25K):**
   - **First-Fit:** Proses 22K akan menempati **blok 30K** karena ini adalah *hole* pertama yang mencukupi untuk 22K (yang 15K dilewati karena kekecilan).
   - **Best-Fit:** Proses 22K akan mencari sisa terkecil, jadi akan masuk ke **blok 25K**. Sisa blok 3K.
   - **Worst-Fit:** Proses 22K mencari *hole* terbesar, jadi akan menempati **blok 40K**. Sisa blok 18K.

2. **Jawaban Kompaksi:** Kompaksi sangat dihindari karena berfokus untuk menyatukan ruang dengan menggeser proses-proses di memori secara fisik. Hal ini menyebabkan **overhead komputasi** yang tinggi karena CPU dan MMU harus mendata ulang register maupun cache dari seluruh relokasi alamat memori yang berpindah.

Terima Kasih

Ada Pertanyaan?

Persiapkan mental di materi Week ke-7 untuk masuk lebih detail tentang lanjutan mekanisme **Virtual Memory** Modern dan Swapping.